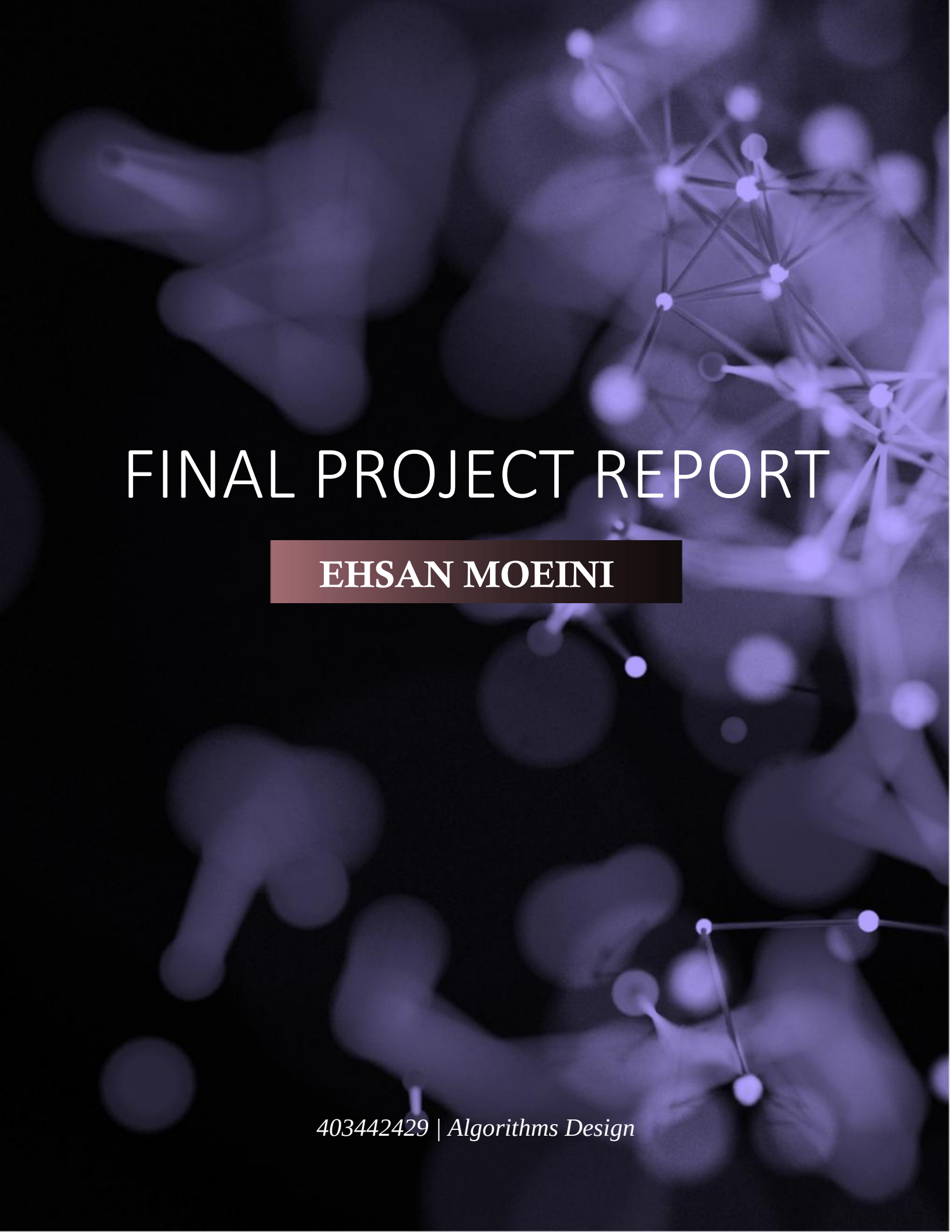


FINAL PROJECT REPORT

EHSAN MOEINI

403442429 | Algorithms Design

INTRODUCTION 3

1. PROBLEM 1: THE TIME-LIMITED EXPEDITION 4

1.1. STATE REPRESENTATION 4
1.2. STEP 1:TREASURE INDEX ASSIGNMENT 4
1.3. STEP 2:INITIAL STATE CONFIGURATION 5
1.4. STEP 3:BFS INITIALIZATION 5
1.5. STEP 4:STATE SPACE EXPLORATION 5
1.6. STEP 5:NEIGHBOR STATE GENERATION 6
1.7. CORRECTNESS REVIEW 6
1.8. COMPLEXITY ANALYSIS 6

2. PROBLEM 2: MAPPING THE RUINS 8

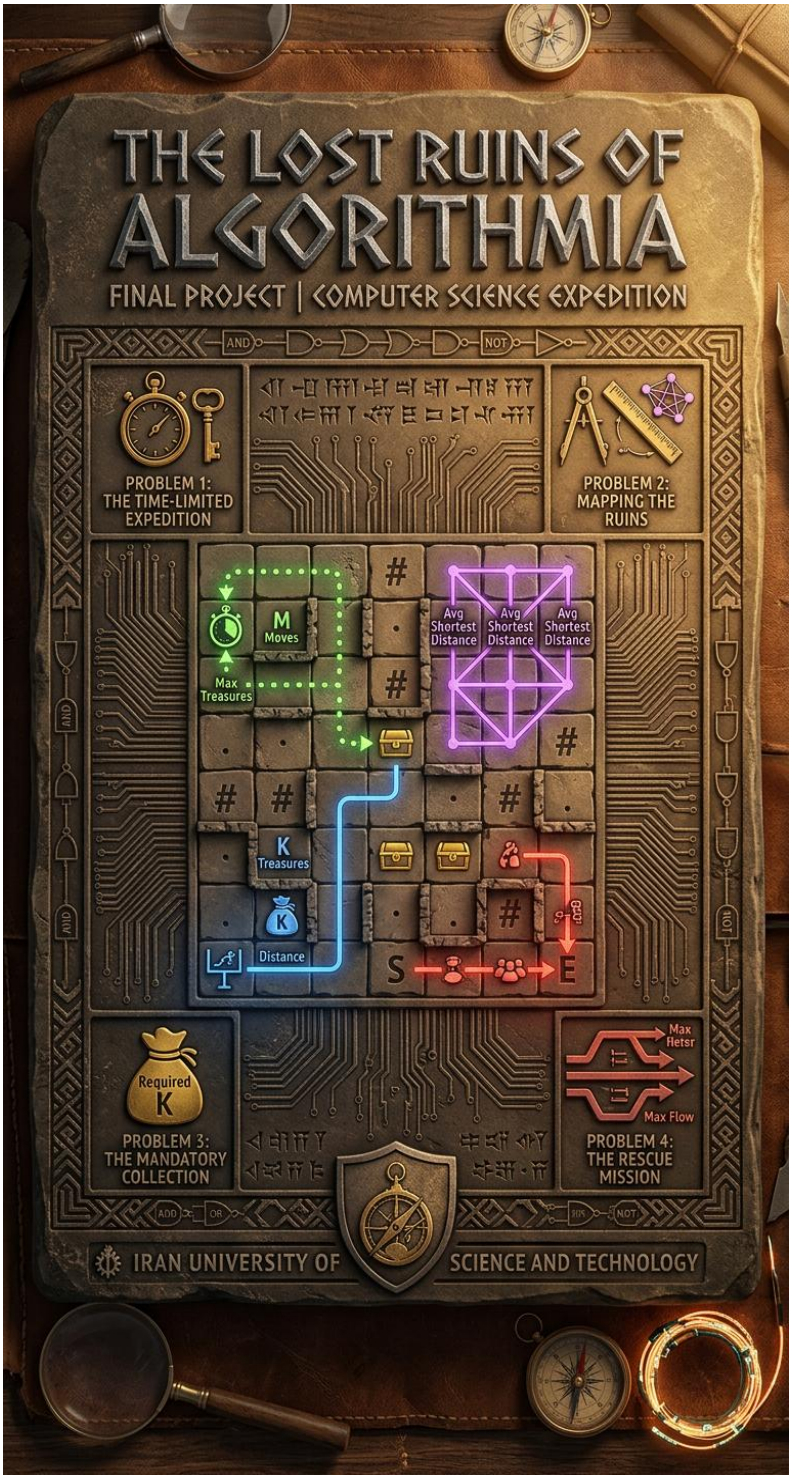
2.1 STEP 1:SINGLE-SOURCE BFS EXECUTION 8
2.2 STEP 2:NEIGHBOR EXPLORATION & DISTANCE TRACKING 8
2.3 STEP 3:ALL-PAIRS SHORTEST PATH AGGREGATION 8
2.4 STEP 4:FINAL AVERAGE METRIC CALCULATION 8
2.5 CORRECTNESS REVIEW 9
2.6 COMPLEXITY ANALYSIS 9

3. PROBLEM 3: THE MANDATORY COLLECTION 10

3.1 3D STATE REPRESENTATION 10
3.2 STEP 1:TREASURE COORDINATES EXTRACTION 10
3.3 STEP 2:DISTANCE DICTIONARY & QUEUE SETUP 10
3.4 STEP 3:BFS LOOP & OBJECTIVE TERMINATION 11
3.5 STEP 4:STATE TRANSITION & AVOIDANCE MECHANICS 11
3.6 COMPLEXITY ANALYSIS 11

4. PROBLEM 4: THE RESCUE MISSION 12

4.1 FLOW NETWORK TRANSFORMATION VIA NODE SPLITTING 12
4.2 STEP 1:BOUNDARY & SPECIAL CASE EVALUATION 12
4.3 STEP 2:VERTEX DUPLICATION MAPPING 13
4.4 STEP 3:RESIDUAL GRAPH CONSTRUCTION 13
4.5 STEP 4: AUGMENTING PATH DISCOVERY (BFS) 14
4.6 STEP 5: FLOW AUGMENTATION & CAPACITY UPDATE 14
4.7 CORRECTNESS REVIEW 15
4.8 COMPLEXITY ANALYSIS 15



Introduction

This report presents the algorithmic solutions for the **Algorithm Design** final project, **"The Lost Ruins of Algorithmia."** As specified in project.pdf , this project provides a challenging and comprehensive framework for applying advanced computer science principles to complex grid manipulation, combinatorial optimization, and graph theory. The project simulates an archaeological expedition within a dynamic 2D grid environment characterized by obstacles, scattered treasures, and strict resource constraints.

This document details the design methodology, technical implementation, and complexity analysis for the four core tasks:

- **The Time-Limited Expedition**
- **Mapping the Ruins**
- **The Mandatory Collection**
- **The Rescue Mission**

The primary objective is to demonstrate the theoretical correctness and runtime efficiency of each developed algorithm.

1. Problem 1: The Time-Limited Expedition

As mentioned in the document of project we use the BFS over a 4-dimensional state space.

1.1 State Representation

```
(row, col, moves_used, treasure_mask)
```

- `row`, `col` represent the current position.
- `moves_used` is the number of moves taken so far.
- `treasure_mask` is a bitmask showing which treasures are collected.

1.2 Step 1: Treasure Index Assignment

For example in the following grid:

```
T . T
. . .
T . .
```

It becomes:

```
T0 . T1
.  .  .
T2 .  .
```

This helps us to represent collected treasures using a bitmask later on.
This part of the code is responsible for this step:

```
treasures = []

for row in range(R):
    for col in range(C):
        if grid[row][col] == 'T':
            treasures.append((row, col))

treasure_index = {}

for idx, (tr, tc) in enumerate(treasures):
    treasure_index[(tr, tc)] = idx
```

1.3 Step 2: Initial State Configuration

If the starting cell of the grid has a treasure, it's collected immediately:

```
start_has_treasure = (sr, sc) in treasure_index
```

Also the relative bit in the mask is turned on:

```
initial_mask = (1 << treasure_index[(sr, sc)])if start_has_treasure
else 0
```

About the logic of the bitmask itself, assume we have 5 treasures and in an optimal path we could gather 3 of them (T0, T1, T4). Showing this in binary format be as `10011` .
Now what the operator `x << y` does, would be like:

$$x \times 2^y$$

```
print(1 << 2) #output: 4
print(1 << 5) #output: 32
```

and by turning it to the binary format:

```
print(bin(1 << 2)) #output: 0b100
print(bin(1 << 5)) #output: 0b100000
```

which each bit corresponds to the relative treasure index being collected or not (1 or 0).

1.4 Step 3: BFS Initialization

```
initial_state = (sr, sc, 0, initial_mask)

queue = deque()
queue.append(initial_state)
```

The BFS queue stores complete info of states. We also use a visited set to prevent viewing states that have already been viewed.

1.5 Step 4: State Space Exploration

At each step of our BFS loop we remove one state from the queue.

```
while queue:
        row, col, moves_used, treasure_mask = queue.popleft()
```

If the current state is at the position of the end cell, it's potentially an answer; So we update our answer:

```
if row == er and col == ec:
        treasures_collected = bin(treasure_mask).count('1')
        max_treasures = max(max_treasures, treasures_collected)
```

And if the current state has run out of moves we skip to the next state in the queue.

1.6 Step 5: Neighbor State Generation

As the most import part of the algorithm we visit neighbors and add unvisited states accordingly.

```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

The direction list contains all of the possible moves (up, down, left, right), which by looping over the current state's neighbors, the new states are obtained.
Just a few conditions to consider while moving in directions:

- Stay inside the grid.
- Do not enter a wall.
- Update the treasure mask if a treasure is collected.

```
new_mask = treasure_mask

if (new_row, new_column) in treasure_index:
  t_idx = treasure_index[(new_row, new_column)]
  new_mask = treasure_mask | (1 << t_idx)
```

The bitwise OR operator `|` ensures the operation over our mask.

1.7 Correctness Review

Each BFS state fully captures:

- Current position
- Number of moves used
- Set of collected treasures

Therefore every feasible path corresponds to exactly one sequence of states.

Since BFS explores all reachable states with at most M moves, every valid path from S to E is considered.

Whenever a state reaches E, the number of collected treasures is evaluated and if it's more than the previous valid values, the max number of treasures will be updated accordingly.
Thus the maximum treasure count among all valid paths is found.

1.8 Complexity Analysis

Time Complexity

The number of possible masks is: 2^t

From each state, we try up to 4 directions, so the BFS work is: $O(4 \times R \times C \times M \times 2^t)$
(R = number of rows, C = number of columns, M = number of moves)
So the Time Complexity becomes: $O(R \times C \times M \times 2^t)$

Space Complexity

The queue and visited structure store states.
So the Space Complexity becomes: $O(R \times C \times M \times 2^t$

Note:

we could've used Dynamic Programing as followed to lower the complexity to:
$O(R \times C \times 2^t)$

```
dp[(row, col, mask)] = minimum moves needed
```

2. Problem 2: Mapping the ruins

Using only the BFS, gives us the shortest distance of two cells of the grid, because BFS is based on exploring level by level, so the first time visiting a cell will always be through the shortest path.

2.1 Step 1: Single-Source BFS Execution

In our `bfs_from_cell` method we run the BFS from one passable cell and find other cell's distance from that base cell.

```
dist = [[-1] * C for _ in range(R)]
dist[start_row][start_col] = 0
```

One 2 dimensional array is considered to demonstrate the whole gird and it's values show the distance from the current cell to each cell corresponding to the grid.
Their initial value is set to `-1` and during the BFS they will be updated.
And as one cell has distance 0 from itself, we set it accordingly in the first place.

2.2 Step 2: Neighbor Exploration & Distance TrackinG

Same as the previous problem we search over the neighbors, consider the conditions of directions and bounds.
Once reaching an unvisited cell (which has value `dist[neighbor_row][neighbor_col] == -1`) we update it's distance on the `dist` array.
Also we keep track of the `total_distance` and the number of reachable cells (`reachable_count`) as we're going to need them for our general formula.

2.3 Step 3: All-Pairs Shortest Path Aggregation

In `solve` method, first we get hold of the `passable_cells` then we run the BFS from each of them to get the total distance (`total_sum`) as well as the total existing paths from one cell to another (`total_count`).

2.4 Step 4:Final Average Metric Calculation

After the BFS is done the average is obtained as followed:

```
average = total_sum / total_count
```

And as we are asked to give the truncated value up to 2 decimal places, the method returns the following value:

```
truncated = math.floor(average * 100) / 100
return f"{truncated:.2f}"
```

2.5 Correctness Review

BFS is the right tool here because all moves have the same cost.
That means BFS always finds the shortest path length from the start cell to every reachable cell.
By running BFS from every passable cell, we compute the shortest distance for every ordered pair of passable cells.
Then we simply average all finite distances.

2.6 Complexity Analysis

Time Complexity

We run BFS from every passable cell.

- One BFS takes $O(R \times C)$
- There can be up to $(R \times C)$ passable cells

Therefore the total time complexity will be: $O((R \times C)^2)$

Space Complexity

The queue and visited structure store states.
Each BFS uses:

- the distance array dist
- The queue

Both take $O(R \times C)$ space.
So the space complexity is: $O(R \times C)$

3. Problem 3: the mandatory collection

In this question we use a 3-dimensional state BFS, as the question itself asked for it; For that we must track the number of moves with another data structure to store the moves for each state.

3.1 3D State Representation

```
(row, col, treasure_mask)
```

- `row`, `col` represent the current position.
- `treasure_mask` is a bitmask showing which treasures are collected.

3.2 Step 1: Treasure Coordinates Extraction

Same as the first problem we start by finding treasures in the grid and representing each with an index assigned to them:

```
treasure_positions = []
for row in range(R):
    for col in range(C):
        if grid[row][col] == 'T':
            treasure_positions.append((row, col))

treasure_index = {}
for idx, (tr, tc) in enumerate(treasure_positions):
    treasure_index[(tr, tc)] = idx
```

3.3 Step 2: Distance Dictionary & Queue Setup

Defining the `moves` dictionary to keep track of the minimum number of moves required to reach to some state. As mentioned previously, the first value that BFS returns, is always obtained through the shortest path as it searches level by level.

```
start_has_treasure = (sr, sc) in treasure_index
initial_mask = (1 << treasure_index[(sr, sc)]) if start_has_treasure
else 0
initial_state = (sr, sc, initial_mask)
moves = {initial_state: 0}

queue = deque()
queue.append(initial_state)
```

3.4 Step 3: BFS Loop & Objective Termination

Everything looks so similar to the first problem except for one difference which is our ending condition.
In our main BFS loop we have:

```
row, col, mask = queue.popleft()

current_moves = moves[(row,col,mask)]
```

And our ending condition would be:

```
treasures_collected = bin(mask).count('1')
if row == er and col == ec and treasures_collected >= K:
    return current_moves
```

So as soon as we gather the desired number of treasures and we hit the end point respectively, the algorithm will be finished and the number moves made, regarding to the current state, will be the answer.

3.5 Step 4:State Transition & Avoidance Mechanics

Every other parts of the code is similar to the first problem.
We also avoid revisiting states and by visiting new ones, we update the number of moves as followed:

```
new_state = (neighbor_row, neighbor_col, new_mask)
if new_state not in moves:
    moves[new_state] = current_moves + 1
    queue.append(new_state)
```

3.6 Complexity Analysis

Time Complexity

The number of possible masks is: 2^t
So the BFS can visit at most $O(R \times C \times 2^t)$ distinct states.
For every state counting set bits takes: $O(T)$

So the Time Complexity becomes $O(R \times C \times T \times 2^t)$ but as $T \leq 10$, we can write: $O(R \times C \times 2^t)$

Space Complexity

The queue and moves structure store 3-dimensional states.
Therefore the maximum memory usage is proportional to the number of states: $O(R \times C \times 2^t)$

4. Problem 4: the rescue mission

we are looking for the maximum number of vertex-disjoint paths from S to E in a grid. As mentioned in the question, we solve this by converting the grid into a flow network and computing a maximum flow.

4.1 Flow Network Transformation via Node Splitting

In a normal grid, the restriction is on **cells** (vertices), not on edges. To model this with flow, we use **node splitting**:

- Every passable cell (r, c) is split into two nodes:

```
node_in(r, c)
```

```
node_out(r, c)
```

- We add an edge from `node_in` to `node_out`:
  - capacity 1 for ordinary cells
  - capacity ∞ for S and E

This ensures that an ordinary cell can be used by at most one path.
Then, for each pair of adjacent passable cells, we add an edge:

- `node_out(u) -> node_in(v)` with capacity ∞

So the only bottleneck is passing through cells.

4.2 Step 1:Boundary & Special Case Evaluation

If the start and end are the same cell, the answer is 1:

```
if sr == er and sc == ec:
return 1
```

If either S or E is blocked, no path exists:

```
if grid[sr][sc] == '#' or grid[er][ec] == '#':
return 0
```

4.3 Step 2: Vertex Duplication Mapping

Each cell (r, c) is mapped to two graph nodes:

```
def node_in(row, col):
        return (row * C + col) * 2

    def node_out(row, col):
        return (row * C + col) * 2 + 1
```

This gives every cell its own internal capacity edge.
The flow starts from the outgoing side of S and ends at the incoming side of E:

```
source = node_out(sr, sc)
sink = node_in(er, ec)
```

4.4 Step 3: Residual Graph Construction

The graph is represented as adjacency lists. Each edge stores:

- destination node
- remaining capacity
- index of the reverse edge

Internal edges

For every passable cell, we add:

`node_in(r, c) -> node_out(r, c)`

with:

- capacity 1 for ordinary cells
- capacity ∞ for S and E

```
for row in range(R):
    for col in range(C):
        if grid[row][col] != '#':
            if (row == sr and col == sc) or (row == er and col == ec):
                cap = float('inf')
            else:
                cap = 1
            add_edge(node_in(row, col), node_out(row, col), cap)
```

Movement edges

For every pair of adjacent passable cells, we add:

```
node_out(u) -> node_in(v)
```

with capacity ∞.

```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
for row in range(R):
    for col in range(C):
        if grid[row][col] != '#':
            u_out = node_out(row, col)
            for dr, dc in directions:
                neighbor_row, neighbor_col = row + dr, col + dc
                if 0 <= neighbor_row < R and 0 <= neighbor_col < C
and grid[neighbor_row][neighbor_col] != '#':
                    v_in = node_in(neighbor_row, neighbor_col)
                    add_edge(u_out, v_in, float('inf'))
```

4.5 Step 4: Augmenting Path Discovery (BFS)

The function bfs_find_path() searches for an augmenting path using BFS.

Edmonds–Karp algorithm always chooses the shortest augmenting path in terms of number of edges.

The BFS stores:

- parent[v]: the previous node on the path
- edge_from[v]: which edge was used to reach v

When the sink is reached, the path is reconstructed and the **bottleneck capacity** is computed.

4.6 Step 5: Flow Augmentation & Capacity Update

Once an augmenting path is found:

- subtract the bottleneck from each forward edge
- add the same amount to each reverse edge

This updates the residual graph so future paths can reuse or reroute flow if needed.

The total flow is increased by the bottleneck value.

```
for u, e_idx in path:
    # Reduce forward edge capacity
    v, cap, rev_idx = graph[u][e_idx]
    graph[u][e_idx][1] -= bottleneck

    # Increase reverse edge capacity
    rev_v, rev_cap, rev_rev_idx = graph[v][rev_idx]
    graph[v][rev_idx][1] += bottleneck

max_flow += bottleneck
```

When no more augmenting paths exist, the current flow is the maximum flow, which equals the maximum number of vertex-disjoint paths.

4.7 Correctness Review

- Every ordinary cell has capacity 1 through its internal edge, so only one path can use it.
- S and E are given infinite internal capacity, so they may be shared.
- Movement between neighboring cells is unrestricted except by cell capacity.
- Therefore, every unit of flow represents one valid path.

So the maximum flow is exactly the maximum number of teams that can travel simultaneously from S to E without conflict.

4.8 Complexity Analysis

Time Complexity

The implementation uses the **Edmonds–Karp** maximum-flow algorithm.

Its worst-case complexity is: $O(V \times E^2)$

Substituting:

$V = O(R \times C)$

$E = O(R \times C)$

gives: $O((R \times C)^3)$ worst-case time complexity.

Space Complexity

The algorithm stores:

- the residual graph,
- BFS parent arrays,
- visited arrays,
- reverse edges.

All of these are proportional to the graph size: $O(V + E)$

which becomes: $O(R \times C)$